

Table of contents

1) Introduction.....	3
2) C PROGRAM STRUCTURE AND EXAMPLES.....	3
a) Basic Hello World Program.....	3
b) Program with User Input.....	3
c) Program with Formatted Output.....	4
3) GCC COMPILATION PROCESS.....	4
a) Installing GCC on Linux.....	4
b) Four Stages of Compilation.....	5
c) Compilation Stage examples.....	5
i) Stage 1: Preprocessing:.....	5
ii) Stage 2: Compilation to Assembly:.....	5
iii) Stage 3: Assembly to Object File:.....	6
iv) Stage 4: Linking:.....	6
4) Return Values and Exit Status.....	6
a) Understanding Return Values.....	6
b) Checking Exit Status.....	7
5) KNOWLEDGE CHECK QUESTIONS.....	7
6) Conclusion.....	9

List of Figures

Listing 1. Basic Hello World Program.....	3
Listing 2. Program with User Input.....	3
Listing 3. Program with Formatted Output.....	4
Listing 4. GCC Installation Commands.....	4
Listing 5. Preprocessing Stage.....	5
Listing 6. Compilation Stage.....	5
Listing 7. Assembly Stage.....	6
Listing 8. Linking Stage.....	6
Listing 9. Program with Return Value.....	6
Listing 10. Checking Program Exit Status.....	7

1) Introduction

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

2) C PROGRAM STRUCTURE AND EXAMPLES

a) Basic Hello World Program

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

A screenshot of a terminal window showing the GNU nano 8.7.1 text editor. The editor is editing a file named 'hello.c'. The code displayed is a basic C program structure: it includes the standard input/output header <stdio.h>, defines a main function, prints 'Hello, World!\n' using printf, and returns 0. The code is as follows:

```
GNU nano 8.7.1 hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Listing 1. Basic Hello World Program

b) Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

A screenshot of a terminal window showing the GNU nano 8.7.1 text editor. The editor is editing a file named 'hello.c'. The code displayed is a C program that prompts the user to enter a number, reads the input using scanf, and then prints the entered number using printf. The code is as follows:

```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number");
    scanf("%d", &number);

    printf("you entered: %d\n", number);
    return 0;
}
```

Listing 2. Program with User Input

c) Program with Formatted Output

Demonstrating multiple format specifiers and variables:

```
GNU nano 8.7.1                                     hello.c *
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m) : ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("age: %d years\n", age);
    printf("height : %.2f m\n", height);
    return 0;
}
```

Listing 3. Program with Formatted Output

3) GCC COMPILATION PROCESS

a) Installing GCC on Linux

The following commands are used to install and verify GCC:

```
bipin@bipin-VirtualBox:~/batch38$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30      linux-image-unsigned-7.0.0-30-generic
  linux-headers-7.0.0-30-generic  linux-main-modules-zfs-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 9
bipin@bipin-VirtualBox:~/batch38$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
```

Listing 4. GCC Installation Commands

b) Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Table 1: The Four Stages of C Compilation

Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers & macros
2. Compiling	gcc -S hello.i -o hello.s	.s file	Generate Assembly code
3. Assembling	gcc -c hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.o -o hello	executable	Final executable

c) Compilation Stage examples

i) Stage 1: Preprocessing:

```
bipin@bipin-VirtualBox:~/batch38$ gcc -E hello.c -o hello.i
bipin@bipin-VirtualBox:~/batch38$ ls
hello.c hello.i hello.o hello.s newfile.txt test test.c
bipin@bipin-VirtualBox:~/batch38$ xat hello.i | less
bipin@bipin-VirtualBox:~/batch38$ cat hello.i | less
```

Listing 5. Preprocessing Stage

ii) Stage 2: Compilation to Assembly:

```
bipin@bipin-VirtualBox:~/batch38$ gcc -S hello.i -o hello.s
hello.c: In function 'main':
hello.c:19:16: warning: format '%d' expects argument of type 'int', but argument 2 has type 'char *' [-Wformat]
   19 |         printf("age: %d years\n", name);
      |                ^~~~~~
      |                |
      |                char *
bipin@bipin-VirtualBox:~/batch38$ cat hello.s
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Enter you name: "
.LC1:
.string "%s"
.LC2:
.string "Enter your age: "
.LC3:
.string "%d"
```

Listing 6. Compilation Stage

iii) Stage 3: Assembly to Object File:

```
4:
bipin@bipin-VirtualBox:~/batch38$ gcc -c hello.s -o hello.o
bipin@bipin-VirtualBox:~/batch38$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
bipin@bipin-VirtualBox:~/batch38$
```

Listing 7. Assembly Stage

iv) Stage 4: Linking:

```
bipin@bipin-VirtualBox:~/batch38$ gcc hello.o -o hello
bipin@bipin-VirtualBox:~/batch38$ ./hello
Enter you name: bipin 19
Enter your age: Enter your height (m) : 6

--- User Info ---
Name: bipin
age: 19 years
height : 6.00 m
bipin@bipin-VirtualBox:~/batch38$
```

Listing 8. Linking Stage

4) Return Values and Exit Status

a) Understanding Return Values

The return value of `main()` indicates program success or failure to the OS:

```
GNU nano 8.7.1                                hello.c *
#include <stdio.h>

int main() {
    int main() {
        printf("Program executed successfully\n");
        return 0; // 0 indicates success
    }
}
```

Listing 9. Program with Return Value

b) Checking Exit Status

```
height : 6.00 m
bipin@bipin-VirtualBox:~/batch38$ nano hello.c
bipin@bipin-VirtualBox:~/batch38$ ./hello
Enter you name: bipin
Enter your age: 19
Enter your height (m) : 6

--- User Info ---
Name: bipin
age: 19 years
height : 6.00 m
bipin@bipin-VirtualBox:~/batch38$ echo $?
0
```

Listing 10. Checking Program Exit Status

5) KNOWLEDGE CHECK QUESTIONS

The following questions assess understanding of Lecture 2 concepts:

- a) What is the purpose of the `#include` directive in C programming? Provide an example of its usage.
- `#include` is a preprocessor directive that tells the compiler to insert the contents of another file into your source file before compilation actually begins.

- b) Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Preprocessing - Handles all directives starting with `#`. file extension `.c` to `.i`

Compilation - Translates the preprocessed C code into assembly language for the target CPU architecture. File extension `.i` to `.s`

Assembly - The assembler converts human-readable assembly instructions into machine code, producing an object file. file extension `.s` to `.o`

Linking - Combines your object file(s) with any needed library code file extension `.o` to executable (no extension on Linux, e.g. `hello`)

- c) What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

0 conventionally tells the operating system (and any calling process, like a shell script) that the program completed successfully, with no errors.

Non-zero (e.g., 1, 2, -1, etc.) signals that something went wrong,

- d) Write a C program that:
- Declares an integer variable
 - Asks the user to input a value
 - Displays the entered value back to the user

```
#include <stdio.h>

int main() {
    int value;

    printf("Enter an integer value: ");
    scanf("%d", &value);

    printf("You entered: %d\n", value);

    return 0;
}
```

- e) What GCC command would you use to:

- 1) Only preprocess a file without compiling?

```
gcc -E hello.c -o hello.i
```

- 2) Generate assembly code but not assemble it?

```
gcc -S hello.i -o hello.s
```

- 3) Create an object file without linking?

```
gcc -c hello.s -o hello.o
```

- f) Explain why C is called a "compile language" and how this differs from interpreted languages.

C is called a compiled language because the entire source code is translated into machine code (via the four stages above) ahead of time. It differs in speed as compiled programs generally run faster since the CPU executes native machine instructions directly, with no interpretation overhead at runtime.

6) Conclusion

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.